Практическое задание №1

Установка необходимых пакетов:

```
!pip install -q tqdm
!pip install --upgrade --no-cache-dir gdown
```

```
Requirement already satisfied: gdown in /usr/local/lib/python3.13/dist-packages (6.5.1)
Requirement already satisfied: beautifulsoup4 in /usr/local/lib/python3.13/dist-packages (from gdown) (4.13.5)
Requirement already satisfied: filelock in /usr/local/lib/python3.13/dist-packages (from gdown) (4.2.7)
Requirement already satisfied: requests[socks] in /usr/local/lib/python3.13/dist-packages (from gdown) (2.32.4)
Requirement already satisfied: tqdm in /usr/local/lib/python3.13/dist-packages (from gdown) (4.67.3)
Requirement already satisfied: urllib3 in /usr/local/lib/python3.13/dist-packages (from gdown) (2.5.0)
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.13/dist-packages (from beautifulsoup4->gdown) (4.0)
Requirement already satisfied: typing-extensions>=4.0.0 in /usr/local/lib/python3.13/dist-packages (from beautifulsoup4->gdc
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.13/dist-packages (from requests[socks]->gd
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.13/dist-packages (from requests[socks]->gdown) (3.20)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.13/dist-packages (from requests[socks]->gdown) (
Requirement already satisfied: PySocks!=1.5.7,>=1.5.6 in /usr/local/lib/python3.13/dist-packages (from requests[socks]->gdow
```

Монтирование Вашего Google Drive к текущему окружению:

```
from google.colab import drive
drive.mount('/content/drive', force_remount=True)
```

```
Mounted at /content/drive
```

Константы, которые пригодятся в коде далее, и ссылки (gdrive идентификаторы) на предоставляемые наборы данных:

```
EVALUATE_ONLY = True
TEST_ON_LARGE_DATASET = True
TISSUE_CLASSES = ('ADI', 'BACK', 'DEB', 'LYM', 'MUC', 'MUS', 'NORM', 'STR', 'TUM')
DATASETS_LINKS = {
    'train': '1XtQzVQ5XbrfxpLHJuL0XBGJ5U7CS-cLi',
    'train_small': '1qd45xXfDwdZjktLFwQb-et-mAaFeCzOR',
    'train_tiny': '1I-2ZOuXLd4QwhZQQltp817Kn3J0Xgbui',
    'test': '1RfPou3pFKpuHDJZ-D9XDFzgvwpUBFlDr',
    'test_small': '1wbRsog0n7uGlHIPGLhyN-PMeT2kdQ2lI',
    'test_tiny': '1viiB0s041CNsAK4itvX8PnYthJ-MDnQc'
}
```

Импорт необходимых зависимостей:

```
from pathlib import Path
import numpy as np
from typing import List
import matplotlib.pyplot as plt
from tqdm.notebook import tqdm
from time import sleep
from PIL import Image
import IPython.display
from sklearn import metrics
from sklearn.metrics import balanced_accuracy_score, ConfusionMatrixDisplay
import gdown

import random
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset as TorchDataset, DataLoader, Subset, random_split
import torchvision.transforms as T

seed = 42
def set_seed(seed = 42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)

set_seed(seed)

if torch.cuda.is_available():
    torch.cuda.manual_seed_all(seed)
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Устройство обучения:", device)
if device.type == "cuda":
    print("Используется следующая видеокарта:", torch.cuda.get_device_name(0))
```

```
Устройство обучения: cuda
Используется следующая видеокарта: Tesla T4
```

Класс Dataset

Предназначен для работы с наборами данных, обеспечивает чтение изображений и соответствующих меток, а также формирование пакетов (батчей).

```
class Dataset:

    def __init__(self, name):
        self.name = name
        self.is_loaded = False
        url = f"https://drive.google.com/uc?export=download&confirm=pbef&id={DATASETS_LINKS[name]}"
        output = f'{name}.npz'
        gdown.download(url, output, quiet=False)
        print(f'Loading dataset {self.name} from npz.')
        np_obj = np.load(f'{name}.npz')
        self.images = np_obj['data']
        self.labels = np_obj['labels']
        self.n_files = self.images.shape[0]
        self.is_loaded = True
        print(f'Done. Dataset {name} consists of {self.n_files} images.')

    def image(self, i):
        # read i-th image in dataset and return it as numpy array
        if self.is_loaded:
            return self.images[i, :, :, :]

    def images_seq(self, n=None):
        # sequential access to images inside dataset (is needed for testing)
        for i in range(self.n_files if not n else n):
            yield self.image(i)

    def random_image_with_label(self):
        # get random image with label from dataset
        i = np.random.randint(self.n_files)
        return self.image(i), self.labels[i]

    def random_batch_with_labels(self, n):
        # create random batch of images with labels (is needed for training)
        indices = np.random.choice(self.n_files, n)
        imgs = [self.image(i) for i in indices]
        labels = np.array([self.labels[i] for i in indices])
        return np.stack(imgs), labels

    def image_with_label(self, i: int):
        # return i-th image with label from dataset
        return self.image(i), self.labels[i]
```

Пример использвания класса Dataset

Загрузим обучающий набор данных, получим произвольное изображение с меткой. После чего визуализируем изображение, выведем метку. В будущем, этот кусок кода можно закомментировать или убрать.

↳ Скрыта 1 ячейка.

Обёртка над Dataset для использования с PyTorch

```
# ============================================================
# (Опционально) Обёртка над Dataset для использования с PyTorch
# ============================================================

# ВНИМАНИЕ:
# Этот код нужен только тем, кто хочет решать задание с помощью PyTorch.
# Он показывает, как "подключить" наш Dataset к torch.utils.data.DataLoader.

try:
    import torch
```

```
    from torch.utils.data import Dataset as TorchDataset, DataLoader
    import torchvision.transforms as T
    from PIL import Image

    class HistologyTorchDataset(TorchDataset):
        """
        Обёртка над Dataset для использования с PyTorch.

        base_dataset: экземпляр Dataset('train'), Dataset('train_small'), etc.
        transform:    функция/объект, преобразующий изображение (PIL.Image -> torch.Tensor).

        """
        def __init__(self, base_dataset, transform=None):
            self.base = base_dataset
            # Минимальный transform по умолчанию:
            # np.uint8 [0, 255] -> float32 [0.0, 1.0]
            self.transform = transform or T.ToTensor()

        def __len__(self):
            # Размер датасета
            return len(self.base.images)

        def __getitem__(self, idx):
            """
            Возвращает (image_tensor, label) для PyTorch.
            image_tensor: torch.Tensor формы [3, H, W]
            label: int
            """
            img, label = self.base.image_with_label(idx)  # img: np.ndarray (H, W, 3)
            img = Image.fromarray(img)                    # в PIL.Image
            img = self.transform(img)                     # в torch.Tensor
            return img, int(label)

except ImportError:
    HistologyTorchDataset = None
    print("PyTorch / torchvision не найдены. Обёртка HistologyTorchDataset недоступна.")
```

Пример использвания класса HistologyTorchDataset

↳ Скрыта 1 ячейка.

Класс Metrics

Реализует метрики точности, используемые для оценивания модели:

1. точность,
2. сбалансированную точность.

```
class Metrics:

    @staticmethod
    def accuracy(gt: List[int], pred: List[int]):
        assert len(gt) == len(pred), 'gt and prediction should be of equal length'
        return sum(int(i[0] == i[1]) for i in zip(gt, pred)) / len(gt)

    @staticmethod
    def accuracy_balanced(gt: List[int], pred: List[int]):
        return balanced_accuracy_score(gt, pred)

    @staticmethod
    def print_all(gt: List[int], pred: List[int], info: str):
        print(f'metrics for {info}:')
        print('\t accuracy {:.4f}:'.format(Metrics.accuracy(gt, pred)))
        print('\t balanced accuracy {:.4f}:'.format(Metrics.accuracy_balanced(gt, pred)))
```

Класс Model

Класс, хранящий в себе всю информацию о модели.

Вам необходимо реализовать методы save, load для сохранения и загрузки модели. Особенно актуально это будет во время тестирования на дополнительных наборах данных.

> *Пожалуйста, убедитесь, что сохранение и загрузка модели работает корректно. Для этого обучите модель, протестируйте, сохраните ее в файл, перезапустите среду выполнения, загрузите обученную модель из файла, вновь протестируйте ее на тестовой выборке и убедитесь в том, что получаемые метрики совпадают с полученными для тестовой выборки ранее.*

Также, Вы можете реализовать дополнительные функции, такие как:

1. валидацию модели на части обучающей выборки;
2. использование кроссвалидации;
3. автоматическое сохранение модели при обучении;
4. загрузку модели с какой-то конкретной итерации обучения (если используется итеративное обучение);
5. вывод различных показателей в процессе обучения (например, значение функции потерь на каждой эпохе);
6. построение графиков, визуализирующих процесс обучения (например, график зависимости функции потерь от номера эпохи обучения);
7. автоматическое тестирование на тестовом наборе/наборах данных после каждой эпохи обучения (при использовании итеративного обучения);
8. автоматический выбор гиперпараметров модели во время обучения;
9. сохранение и визуализацию результатов тестирования;
10. Использование аугментации и других способов синтетического расширения набора данных (дополнительным плюсом будет обоснование необходимости и обоснование выбора конкретных типов аугментации)
11. и т.д.

Полный список опций и дополнений приведен в презентации с описанием задания.

При реализации дополнительных функций допускается добавление параметров в существующие методы и добавление новых методов в класс модели.

```
class BaseClassifier(nn.Module):
    """
    #LBL1
    Простой классификатор с настроенным пайплайном
    """

    def __init__(self, num_classes = len(TISSUE_CLASSES)):
        super().__init__()
        self.num_classes = num_classes
        self.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
        self.history = {}
        self.to(self.device)
    #LBL2
    def _run_epoch(self, loader, criterion, optimizer = None):
        is_training = optimizer is not None
        self.train(is_training)
        total_loss = 0.0
        total_correct = 0
        total_count = 0

        for images, labels in loader:
            images = images.to(self.device, non_blocking = True)
            labels = labels.to(self.device, dtype=torch.long, non_blocking=True)

            if is_training: #обнуляем градиент для каждого батча
                optimizer.zero_grad(set_to_none=True)

            with torch.set_grad_enabled(is_training):
                logits = self(images) #запуск расчета логитов
                loss = criterion(logits, labels)
                if is_training:
                    loss.backward()
                    optimizer.step()
            batch_size = labels.size(0)
            total_loss += loss.item()*batch_size
            total_correct += (logits.argmax(dim=1) == labels).sum().item()
            total_count += batch_size
        epoch_loss = total_loss / total_count
        epoch_accuracy = total_correct / total_count
        return epoch_loss, epoch_accuracy

    #LBL3
    def _fit(self, train_loader: DataLoader, val_loader: DataLoader, optimizer,
             criterion, epochs, scheduler=None, patience=None):
        #намеренно переимовал train на fit иначе был бы конфликт имен с методом train из pytorch
        history = {
            "train_loss": [],
            "val_loss": [],
            "train_accuracy": [],
```

```
            "val_accuracy": [],
            "learning_rate": [],
        }
        best_val_loss = float("inf")
        best_state = None
        best_epoch = 0
        epochs_without_improvement = 0

        print(f"Обучение на устройстве: {self.device}")

        for epoch in range(epochs):
            train_loss, train_accuracy = self._run_epoch(train_loader, criterion, optimizer)
            val_loss, val_accuracy = self._run_epoch(val_loader, criterion, optimizer=None)

            if scheduler is not None:
                scheduler.step(val_loss) #передаем планировщику потери

            current_lr = optimizer.param_groups[0]["lr"] #запомнинаем текущую lr что корректировать её в процессе обучения
            history["train_loss"].append(train_loss)
            history["val_loss"].append(val_loss)
            history["train_accuracy"].append(train_accuracy)
            history["val_accuracy"].append(val_accuracy)
            history["learning_rate"].append(current_lr)
            print(
                f"Эпоха {epoch}/{epochs} | "
                f"train loss: {train_loss:.4f} | "
                f"val loss: {val_loss:.4f} | "
                f"train acc: {train_accuracy:.4f} | "
                f"val acc: {val_accuracy:.4f} | "
                f"lr: {current_lr:.6f}"
            )
            if val_loss < best_val_loss:
                best_val_loss = val_loss
                best_epoch = epoch + 1
                best_state = {} #словарь для сохранения лучших параметров
                for key, value in self.state_dict().items():
                    value = value.detach().cpu().clone()
                    best_state[key] = value
                epochs_without_improvement = 0
            else:
                epochs_without_improvement += 1
            if patience is not None and epochs_without_improvement >= patience:
                print("Модель перестала обучаться, досрочное завершение")
                break
        if best_state is not None:
            self.load_state_dict(best_state)
        history["best_epoch"] = best_epoch
        self.history = history
        self.eval()
        print(f"Обучение завершено. Лучшая эпоха: {best_epoch}.")
        return history

    def save(self, name: str):
        # todo: сохраните веса вашей модели.
        # Пример уже рабочий: он пишет npz на Google Drive, а load читает тот же файл.
        path = f'/content/drive/MyDrive/{name}.pth'
        torch.save(self.state_dict(), path)
        print(f'Модель сохранена в {path}.')
        #print('Чтобы сдать файл ссылкой, откройте к нему доступ и вставьте id в load().')

    def load(self, name: str):
        # todo: если сдаёте модель ссылкой, замените None на id вашего файла.
        # Пока id не указан, читаетсагя файл, который save записал на примонтированный Drive.
        name_to_id_dict = {
            'best': "1TpWZyzKipzBZv-XUgmTBjv2JdjZp3mcQ",
            'baseline': "1bzczlNWmt04c0Rmek2l6gpDqfWqt_pLk" #здесь хранятся тестовой базовой модели
        }
        file_id = name_to_id_dict.get(name)
        if file_id:
            path = Path(f"/content/{name}.pth")
            gdown.download(id=file_id, output=str(path), quiet=False)
        else:
            path = Path(f'/content/drive/MyDrive/{name}.pth')
        if not path.exists():
            raise FileNotFoundError(
                f'Не найден {path}. Вызовите save("{name}") при смонтированном Google Drive '
                f'или укажите id файла в name_to_id_dict внутри load().'
            )
        state_dict = torch.load(path, map_location=self.device)
        self.load_state_dict(state_dict)
        self.to(self.device)
        self.eval()
```

```
            print(f"Веса модели загружены из {path}")

        #LBL4
        def test_on_dataset(self, dataset: Dataset, limit=None, seed=42):
            predictions = []
            n = dataset.n_files if not limit else int(dataset.n_files * limit)
            test_ds = HistologyTorchDataset(dataset, transform=T.ToTensor())
            generator = torch.Generator().manual_seed(seed)
            shuffled_indices = torch.randperm(len(test_ds), generator=generator)[:n].tolist()
            test_ds = Subset(test_ds, shuffled_indices) #создаем подмножество датасета
            test_loader = DataLoader(
                test_ds,
                batch_size = 64,
                shuffle=False,
                pin_memory=(self.device.type == "cuda")
                #используем специальную область озу для ускорения передачи
                #данных с озу на gpu
            )
            self.eval()
            with torch.inference_mode():
                for images, _labels in tqdm(test_loader, total=len(test_loader)):
                    images = images.to(self.device, non_blocking=True)
                    logits = self(images)
                    predictions.extend(logits.argmax(dim=1).cpu().tolist())

            return predictions, shuffled_indices

        #LBL5
        def test_on_image(self, img: np.ndarray, submission_year: int):
            image = Image.fromarray(img)
            image_tensor = T.ToTensor()(image).unsqueeze(0).to(self.device)
            self.eval()
            with torch.inference_mode():
                logits = self(image_tensor)
                prediction = int(logits.argmax(dim=1).item())

            return prediction
```

```
    class BaselineModel(BaseClassifier):
        """
        Самая простая CNN с полным пайплайном
        """
        def __init__(self, num_classes=len(TISSUE_CLASSES)):
            super().__init__(num_classes)
            self.conv1 = nn.Conv2d(3, 6, kernel_size=5)
            self.pool = nn.MaxPool2d(kernel_size=4, stride=4)
            self.conv2 = nn.Conv2d(6, 12, kernel_size=5)
            # Для входа 224x224:
            # 224 -> conv1: 220 -> pool: 55 -> conv2: 51 -> pool: 12.
            self.fc1 = nn.Linear(12 * 12 * 12, num_classes)
            self.to(self.device)

        def forward(self, x):
            x = self.pool(torch.relu(self.conv1(x)))
            x = self.pool(torch.relu(self.conv2(x)))
            x = x.flatten(1)
            return self.fc1(x)
        #LBL6
        def fit(self, dataset, epochs=10, batch_size=32, lr=1e-3, val_split=0.2, patience=4):
            set_seed(seed)
            full_ds = HistologyTorchDataset(dataset, transform=T.ToTensor())
            val_size = int(len(full_ds) * val_split)
            train_size = len(full_ds) - val_size
            train_ds, val_ds = random_split(
                full_ds,
                [train_size, val_size],
                generator=torch.Generator().manual_seed(seed)
            )

            train_loader = DataLoader(
                train_ds,
                batch_size=batch_size,
                shuffle=True,
                generator=torch.Generator().manual_seed(seed),
                pin_memory=(self.device.type == 'cuda'),
            )
            val_loader = DataLoader(
                val_ds,
                batch_size=batch_size,
                shuffle=False,
                pin_memory=(self.device.type == 'cuda'),
```

```
        )
        criterion = nn.CrossEntropyLoss()
        optimizer = optim.SGD(
            self.parameters(),
            lr=lr,
            momentum=0.9
        )

        return self._fit(
            train_loader, val_loader, optimizer, criterion,
            epochs=epochs, scheduler=None, patience=patience
        )
```

```
#LBL7
class RandomQuarterTurn:
    """Случайный поворот тензора
        на 0, 90, 180 или 270 градусов
    """

    def __call__(self, image):
        k = int(torch.randint(0, 4, (1,)).item())

        return torch.rot90(
            image,
            k=k,
            dims=(-2, -1)
        ).contiguous()
```

```
class Model(BaseClassifier):
    """
    Улучшенная модель:
    1. Больше сверточных фильтров
    2. Использование BatchNorm
    3. Dropout для уменьшения переобучения
    """
    #LBL8
    def __init__(self, num_classes=len(TISSUE_CLASSES)):
        super().__init__(num_classes)
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, padding=1, stride=2, bias=False),
            nn.BatchNorm2d(32),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2),

            nn.Conv2d(32, 64, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),

            nn.Conv2d(64, 64, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2),
            nn.Dropout2d(0.15),

            nn.Conv2d(64, 128, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),
            nn.AdaptiveAvgPool2d((4, 4)),
        )

        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Dropout(0.30),
            nn.Linear(128*4*4, num_classes)
        )
        self.to(self.device)

    def forward(self, x):
        x = self.features(x)
        return self.classifier(x)

    def fit(self, dataset: Dataset, epochs=20, batch_size=32,
            lr=3e-4, weight_decay=1e-4, val_split=0.2, patience=5):
        """
        Использовал аугментацию
        Использование оптимизатора AdamW и L2 регуляризации
        Динамическое изменение lr с помощью scheduler
        Досрочное завершение обучения при отсутствии прогресса
        """
        set_seed(seed)
```

```
        train_transform = T.Compose([
            T.ToTensor(),
            T.RandomHorizontalFlip(p=0.5),
            T.RandomVerticalFlip(p=0.5),

            RandomQuarterTurn(),
        ])
        val_transform = T.ToTensor()

        indices = torch.randperm( #рандомно перемешиваем индексы
            len(dataset.images),
            generator=torch.Generator().manual_seed(seed)
        ).tolist()

        val_size = int(len(indices) * val_split)
        train_indices = indices[val_size:]
        val_indices = indices[:val_size]

        train_base = HistologyTorchDataset(dataset, transform=train_transform)
        val_base = HistologyTorchDataset(dataset, transform=val_transform)

        train_ds = Subset(train_base, train_indices)
        val_ds = Subset(val_base, val_indices)

        train_loader = DataLoader(
            train_ds,
            batch_size=batch_size,
            shuffle=True,
            generator=torch.Generator().manual_seed(seed),
            pin_memory=(self.device.type == "cuda"),
        )
        val_loader = DataLoader(
            val_ds,
            batch_size=batch_size,
            shuffle=False,
            pin_memory=(self.device.type == "cuda"),
        )

        criterion = nn.CrossEntropyLoss(label_smoothing=0.05) #label_smoothing для смягчения итоговых вероятностей

        optimizer = optim.AdamW(
            self.parameters(),
            lr=lr,
            weight_decay=weight_decay,
        )
        scheduler = optim.lr_scheduler.ReduceLROnPlateau(
           optimizer,
           mode="min",
           factor=0.5,
           patience=2
        )

        return self._fit(
            train_loader, val_loader, optimizer, criterion,
            epochs, scheduler, patience
        )
```

Классификация изображений

Используя введенные выше классы можем перейти уже непосредственно к обучению модели классификации изображений. Пример общего пайплайна решения задачи приведен ниже. Вы можете его расширять и улучшать. В данном примере используются наборы данных 'train_small' и 'test_small'.

```
d_train = Dataset('train')
d_test = Dataset('test')
```

```
Downloading...
From (original): https://drive.google.com/uc?id=1XtQzVQ5XbrfxpLHJuL0XBGJ5U7CS-cLi
From (redirected): https://drive.google.com/uc?id=1XtQzVQ5XbrfxpLHJuL0XBGJ5U7CS-cLi&confirm=t&uuid=1de5b998-9d2d-4edc-9660-5
To: /content/train.npz
100%|██████████| 2.10G/2.10G [00:33<00:00, 62.3MB/s]
Loading dataset train from npz.
Done. Dataset train consists of 18000 images.
Downloading...
From (original): https://drive.google.com/uc?id=1RfPou3pFKpuHDJZ-D9XDFzgvwpUBFlDr
From (redirected): https://drive.google.com/uc?id=1RfPou3pFKpuHDJZ-D9XDFzgvwpUBFlDr&confirm=t&uuid=75911695-4bd7-40d1-a64c-6
To: /content/test.npz
```

```
100%|██████████| 525M/525M [00:07<00:00, 67.9MB/s]
Loading dataset test from npz.
Done. Dataset test consists of 4500 images.
```

```
#LBL9

set_seed(seed)
baseline_model = BaselineModel()

if not EVALUATE_ONLY:
    baseline_history = baseline_model.fit(
        d_train,
        epochs=10,
        batch_size=32,
        lr=1e-3,
        val_split=0.2,
        patience=4,
    )

    baseline_model.save('baseline')

else:
    baseline_model.load('baseline')
```

```
Обучение на устройстве: cuda
Эпоха 0/10 | train loss: 2.1628 | val loss: 2.0964 | train acc: 0.1744 | val acc: 0.2933 | lr: 0.001000
Эпоха 1/10 | train loss: 1.7397 | val loss: 1.3647 | train acc: 0.3658 | val acc: 0.4689 | lr: 0.001000
Эпоха 2/10 | train loss: 1.2703 | val loss: 1.2027 | train acc: 0.5112 | val acc: 0.5469 | lr: 0.001000
Эпоха 3/10 | train loss: 1.1464 | val loss: 1.3359 | train acc: 0.5622 | val acc: 0.5033 | lr: 0.001000
Эпоха 4/10 | train loss: 1.0886 | val loss: 1.0441 | train acc: 0.5878 | val acc: 0.6156 | lr: 0.001000
Эпоха 5/10 | train loss: 1.0111 | val loss: 1.0390 | train acc: 0.6240 | val acc: 0.5967 | lr: 0.001000
Эпоха 6/10 | train loss: 0.9391 | val loss: 0.9155 | train acc: 0.6560 | val acc: 0.6739 | lr: 0.001000
Эпоха 7/10 | train loss: 0.8672 | val loss: 0.8292 | train acc: 0.6874 | val acc: 0.7039 | lr: 0.001000
Эпоха 8/10 | train loss: 0.8181 | val loss: 0.8849 | train acc: 0.7059 | val acc: 0.6892 | lr: 0.001000
Эпоха 9/10 | train loss: 0.7697 | val loss: 0.9068 | train acc: 0.7292 | val acc: 0.6681 | lr: 0.001000
Обучение завершено. Лучшая эпоха: 8.
Модель сохранена в /content/drive/MyDrive/baseline.pth.
```

Пример тестирования модели на части набора данных:

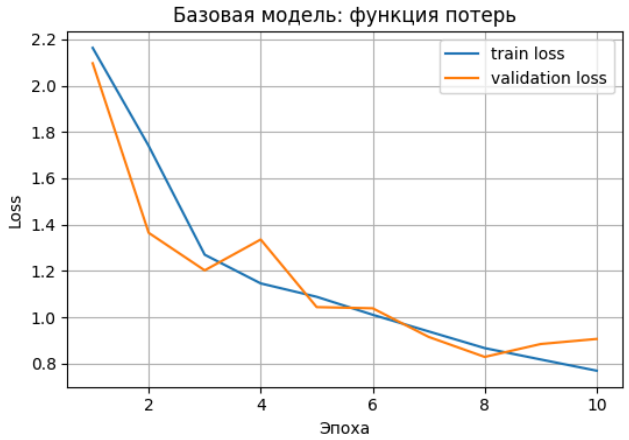
```
#LBL10
if not EVALUATE_ONLY:
    epochs_axis = range(1, len(baseline_history["train_loss"]) + 1)

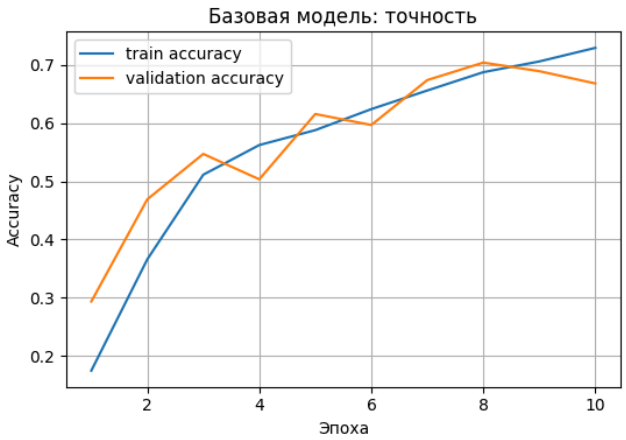
    fig, axes = plt.subplots(1, 2, figsize=(11, 4))

    axes[0].plot(epochs_axis, baseline_history["train_loss"], label="train loss")
    axes[0].plot(epochs_axis, baseline_history["val_loss"], label="validation loss")
    axes[0].set_title("Базовая модель: функция потерь")
    axes[0].set_xlabel("Эпоха")
    axes[0].set_ylabel("Loss")
    axes[0].legend()
    axes[0].grid(True)

    axes[1].plot(epochs_axis, baseline_history["train_accuracy"], label="train accuracy")
    axes[1].plot(epochs_axis, baseline_history["val_accuracy"], label="validation accuracy")
    axes[1].set_title("Базовая модель: точность")
    axes[1].set_xlabel("Эпоха")
    axes[1].set_ylabel("Accuracy")
    axes[1].legend()
    axes[1].grid(True)

    plt.tight_layout()
    plt.show()
else:
    print("Графики обучения появятся только после запуска обучения с EVALUATE_ONLY=False.")
```



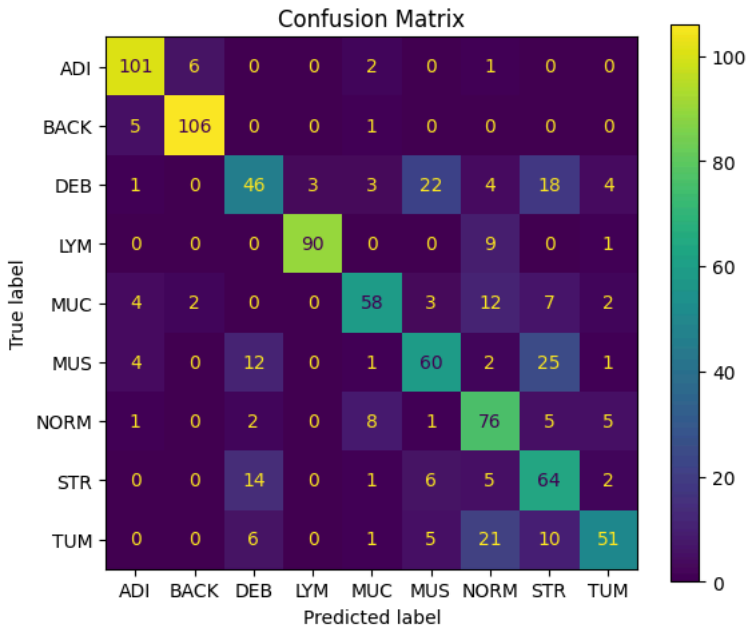



```
#LBL11
pred_1, test_indices = baseline_model.test_on_dataset(d_test, limit=0.2, seed=seed)
true_labels = np.asarray(d_test.labels)[test_indices]
Metrics.print_all(
    true_labels.tolist(),
    pred_1,
    'baseline: 50% of test'
)
```

```
100%                                                15/15 [00:01<00:00, 12.42it/s]
metrics for baseline: 50% of test:
     accuracy 0.7244:
     balanced accuracy 0.7183:
```

```
#LBL12
cm = metrics.confusion_matrix(true_labels.tolist(), pred_1)
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
                              display_labels=TISSUE_CLASSES)

fig, ax = plt.subplots(figsize=(6, 5))
disp.plot(ax=ax)
ax.set_title("Confusion Matrix")
plt.tight_layout()
plt.show()
```



Запуск и тестирование итоговой модели

```
set_seed(seed)
model = Model()

if not EVALUATE_ONLY:
```

```
    improved_history = model.fit(
        d_train,
        epochs=30,
        batch_size=128,
        lr=6e-4,
        weight_decay=1e-4,
        val_split=0.2,
        patience=5,
    )
    model.save('best')
else:
    model.load('best')
```

```
Обучение на устройстве: cuda
Эпоха 0/30 | train loss: 1.1091 | val loss: 1.2392 | train acc: 0.6552 | val acc: 0.6358 | lr: 0.000600
Эпоха 1/30 | train loss: 0.8737 | val loss: 1.2223 | train acc: 0.7663 | val acc: 0.6022 | lr: 0.000600
Эпоха 2/30 | train loss: 0.7890 | val loss: 0.7488 | train acc: 0.8042 | val acc: 0.8158 | lr: 0.000600
Эпоха 3/30 | train loss: 0.7193 | val loss: 0.7877 | train acc: 0.8330 | val acc: 0.8178 | lr: 0.000600
Эпоха 4/30 | train loss: 0.6831 | val loss: 0.9499 | train acc: 0.8516 | val acc: 0.7372 | lr: 0.000600
Эпоха 5/30 | train loss: 0.6339 | val loss: 0.9951 | train acc: 0.8731 | val acc: 0.7064 | lr: 0.000300
Эпоха 6/30 | train loss: 0.5834 | val loss: 0.7091 | train acc: 0.8950 | val acc: 0.8347 | lr: 0.000300
Эпоха 7/30 | train loss: 0.5609 | val loss: 0.5216 | train acc: 0.9065 | val acc: 0.9286 | lr: 0.000300
Эпоха 8/30 | train loss: 0.5555 | val loss: 0.6038 | train acc: 0.9101 | val acc: 0.8750 | lr: 0.000300
Эпоха 9/30 | train loss: 0.5416 | val loss: 0.5036 | train acc: 0.9161 | val acc: 0.9336 | lr: 0.000300
Эпоха 10/30 | train loss: 0.5273 | val loss: 0.5127 | train acc: 0.9200 | val acc: 0.9272 | lr: 0.000300
Эпоха 11/30 | train loss: 0.5203 | val loss: 0.5160 | train acc: 0.9203 | val acc: 0.9289 | lr: 0.000300
Эпоха 12/30 | train loss: 0.5117 | val loss: 0.7970 | train acc: 0.9263 | val acc: 0.8383 | lr: 0.000150
Эпоха 13/30 | train loss: 0.4906 | val loss: 0.5091 | train acc: 0.9359 | val acc: 0.9158 | lr: 0.000150
Эпоха 14/30 | train loss: 0.4857 | val loss: 0.4975 | train acc: 0.9382 | val acc: 0.9350 | lr: 0.000150
Эпоха 15/30 | train loss: 0.4881 | val loss: 0.4719 | train acc: 0.9370 | val acc: 0.9397 | lr: 0.000150
Эпоха 16/30 | train loss: 0.4781 | val loss: 0.4809 | train acc: 0.9408 | val acc: 0.9339 | lr: 0.000150
Эпоха 17/30 | train loss: 0.4764 | val loss: 0.4459 | train acc: 0.9399 | val acc: 0.9536 | lr: 0.000150
Эпоха 18/30 | train loss: 0.4708 | val loss: 0.4201 | train acc: 0.9430 | val acc: 0.9636 | lr: 0.000150
Эпоха 19/30 | train loss: 0.4690 | val loss: 0.4344 | train acc: 0.9443 | val acc: 0.9572 | lr: 0.000150
Эпоха 20/30 | train loss: 0.4638 | val loss: 0.4150 | train acc: 0.9464 | val acc: 0.9619 | lr: 0.000150
Эпоха 21/30 | train loss: 0.4633 | val loss: 0.4196 | train acc: 0.9443 | val acc: 0.9592 | lr: 0.000150
Эпоха 22/30 | train loss: 0.4643 | val loss: 0.4346 | train acc: 0.9447 | val acc: 0.9531 | lr: 0.000150
Эпоха 23/30 | train loss: 0.4614 | val loss: 0.4463 | train acc: 0.9453 | val acc: 0.9469 | lr: 0.000075
Эпоха 24/30 | train loss: 0.4498 | val loss: 0.4039 | train acc: 0.9511 | val acc: 0.9711 | lr: 0.000075
Эпоха 25/30 | train loss: 0.4455 | val loss: 0.4011 | train acc: 0.9520 | val acc: 0.9644 | lr: 0.000075
Эпоха 26/30 | train loss: 0.4478 | val loss: 0.4035 | train acc: 0.9509 | val acc: 0.9678 | lr: 0.000075
Эпоха 27/30 | train loss: 0.4422 | val loss: 0.3960 | train acc: 0.9549 | val acc: 0.9717 | lr: 0.000075
Эпоха 28/30 | train loss: 0.4452 | val loss: 0.4089 | train acc: 0.9510 | val acc: 0.9678 | lr: 0.000075
Эпоха 29/30 | train loss: 0.4444 | val loss: 0.4112 | train acc: 0.9513 | val acc: 0.9686 | lr: 0.000075
Обучение завершено. Лучшая эпоха: 28.
Модель сохранена в /content/drive/MyDrive/best.pth.
```

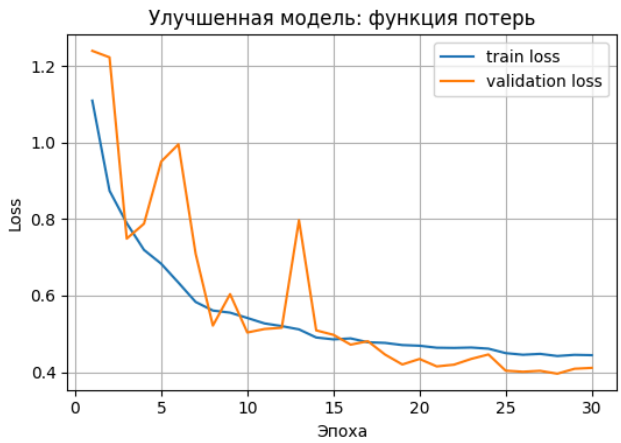
```
if not EVALUATE_ONLY:
    epochs_axis = range(1, len(improved_history["train_loss"]) + 1)

    fig, axes = plt.subplots(1, 2, figsize=(11, 4))

    axes[0].plot(epochs_axis, improved_history["train_loss"], label="train loss")
    axes[0].plot(epochs_axis, improved_history["val_loss"], label="validation loss")
    axes[0].set_title("Улучшенная модель: функция потерь")
    axes[0].set_xlabel("Эпоха")
    axes[0].set_ylabel("Loss")
    axes[0].legend()
    axes[0].grid(True)

    axes[1].plot(epochs_axis, improved_history["train_accuracy"], label="train accuracy")
    axes[1].plot(epochs_axis, improved_history["val_accuracy"], label="validation accuracy")
    axes[1].set_title("Улучшенная модель: точность")
    axes[1].set_xlabel("Эпоха")
    axes[1].set_ylabel("Accuracy")
    axes[1].legend()
    axes[1].grid(True)

    plt.tight_layout()
    plt.show()
```





Чтобы изменить содержимое ячейки, дважды нажмите на нее (или выберите "Ввод")

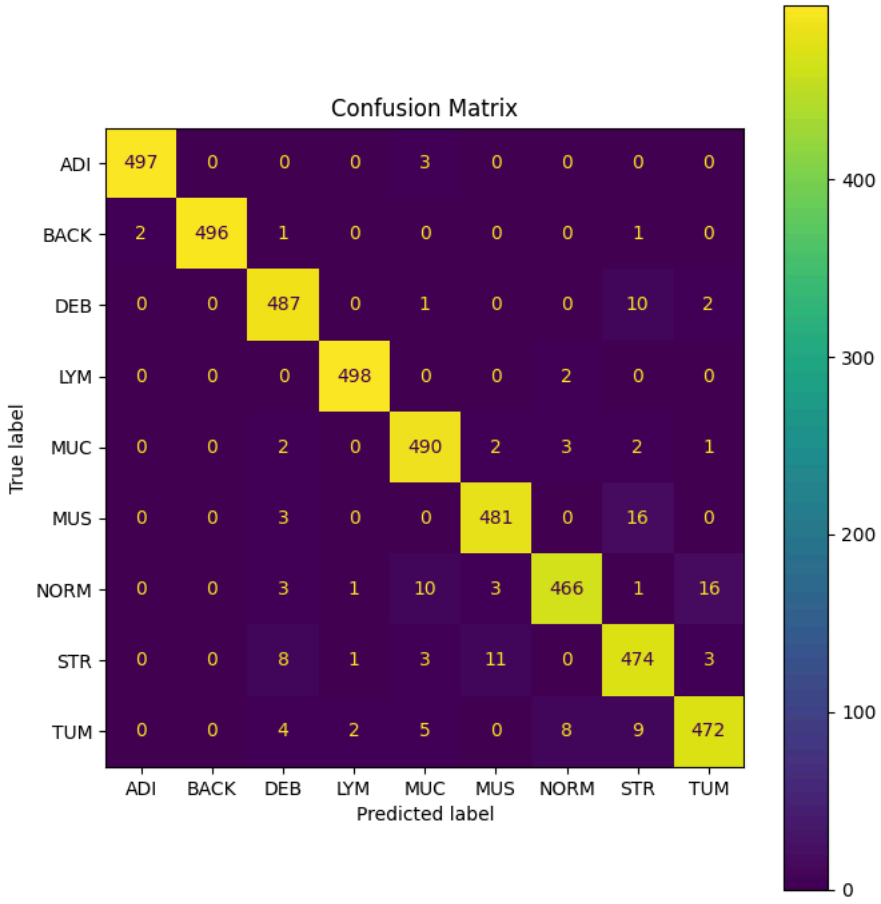
Пример тестирования модели на полном наборе данных:

```
# evaluating model on full test dataset (may take time)
if TEST_ON_LARGE_DATASET:
    pred_1_improved, test_indices = model.test_on_dataset(d_test, limit=1, seed=seed)
    true_labels = np.asarray(d_test.labels)[test_indices]
    Metrics.print_all(
        true_labels.tolist(),
        pred_1_improved,
        'model: full test'
    )

    cm = metrics.confusion_matrix(true_labels.tolist(), pred_1_improved)
    disp = ConfusionMatrixDisplay(confusion_matrix=cm,
                                  display_labels=TISSUE_CLASSES)

    fig, ax = plt.subplots(figsize=(7, 7))
    disp.plot(ax=ax)
    ax.set_title("Confusion Matrix")
    plt.tight_layout()
    plt.show()
```

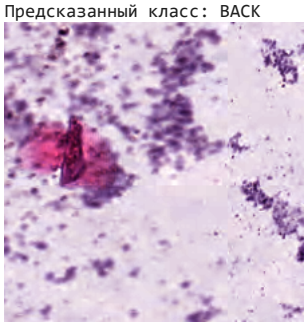
```
100%                                                 71/71 [00:06<00:00, 12.70it/s]
metrics for model: full test:
     accuracy 0.9691:
     balanced accuracy 0.9691:
```



Результат предсказания для рандомного изображения

```
pos = np.random.randint(len(pred_1_improved))
idx = test_indices[pos]
img = d_test.images[idx]
true_label = int(d_test.labels[idx])
predicted_label = int(pred_1_improved[pos])
true_class = TISSUE_CLASSES[true_label]
predicted_class = TISSUE_CLASSES[predicted_label]
print(f"Индекс: {idx}")
print(f"Реальный класс: {true_class}")
print(f"Предсказанный класс: {predicted_class}")
pil_img = Image.fromarray(img)
IPython.display.display(pil_img)
```

```
Индекс: 602
Реальный класс: BACK
Предсказанный класс: BACK
```



Результат работы пайплайна обучения и тестирования выше тоже будет оцениваться. Поэтому не забудьте присылать на проверку ноутбук с выполненными ячейками кода с демонстрациями метрик обучения, графиками и т.п. В этом пайплайне Вам необходимо продемонстрировать работу всех реализованных дополнений, улучшений и т.п.

Настоятельно рекомендуется после получения пайплайна с полными результатами обучения экспортировать ноутбук в pdf (файл -> печать) и прислать этот pdf вместе с самим ноутбуком.

Тестирование модели на других наборах данных

Ваша модель должна поддерживать тестирование на других наборах данных. Для удобства, Вам предоставляется набор данных test_tiny, который представляет собой малую часть (2% изображений) набора test. Ниже приведен фрагмент кода, который будет осуществлять тестирование для оценивания Вашей модели на дополнительных тестовых наборах данных.

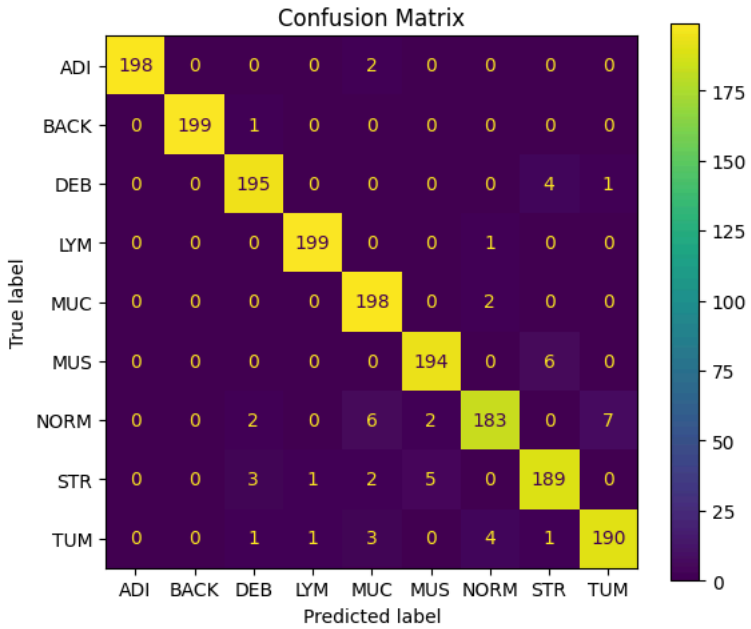
Прежде чем отсылать задание на проверку, убедитесь в работоспособности фрагмента кода ниже.

```
#LBL13
final_model = Model()
final_model.load('best')
d_test_tiny = Dataset('test_small')
pred, test_indices = final_model.test_on_dataset(d_test_tiny)
true_labels = np.asarray(d_test_tiny.labels)[test_indices]
Metrics.print_all(true_labels.tolist(), pred, 'test-small')


cm = metrics.confusion_matrix(true_labels.tolist(), pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
                              display_labels=TISSUE_CLASSES)

fig, ax = plt.subplots(figsize=(6, 5))
disp.plot(ax=ax)
ax.set_title("Confusion Matrix")
plt.tight_layout()
plt.show()
```

```
Downloading...
From: https://drive.google.com/uc?id=1a8LElUsk00x2KgJm_utLA-Dy7Tur_EGq
To: /content/best.pth
100%|██████████| 606k/606k [00:00<00:00, 113MB/s]
Веса модели загружены из /content/best.pth
Downloading...
From (original): https://drive.google.com/uc?id=1wbRsog0n7uGlHIPGLhyN-PMeT2kdQ2lI
From (redirected): https://drive.google.com/uc?id=1wbRsog0n7uGlHIPGLhyN-PMeT2kdQ2lI&confirm=t&uuid=661792db-c066-4786-aa98-6
To: /content/test_small.npz
100%|██████████| 211M/211M [00:01<00:00, 156MB/s]
Loading dataset test_small from npz.
Done. Dataset test_small consists of 1800 images.
100%                                                  29/29 [00:02<00:00, 11.55it/s]
metrics for test-small:
     accuracy 0.9694:
     balanced accuracy 0.9694:
```



Google Drive пока не отключаем: примеры архивов ниже пишут файлы на Диск. Отключение стоит в последней ячейке.

```
# Диск остаётся примонтированным до последней ячейки.
```

Дополнительные "полезности"

Ниже приведены примеры использования различных функций и библиотек, которые могут быть полезны при выполнении данного практического задания.

Измерение времени работы кода

Измерять время работы какой-либо функции можно легко и непринужденно при помощи функции timeit из соответствующего модуля:

↳ Скрыта 1 ячейка.

Scikit-learn

Для использования "классических" алгоритмов машинного обучения рекомендуется использовать библиотеку scikit-learn (https://scikit-learn.org/stable/). Пример классификации изображений цифр из набора данных MNIST при помощи классификатора SVM:

↳ Скрыта 1 ячейка.

Scikit-image

Реализовывать различные операции для работы с изображениями можно как самостоятельно, работая с массивами numpy, так и используя специализированные библиотеки, например, scikit-image (https://scikit-image.org/). Ниже приведен пример использования Canny edge detector.

↳ Скрыта 1 ячейка.

Tensorflow 2

Для создания и обучения нейросетевых моделей можно использовать фреймворк глубокого обучения Tensorflow 2. Ниже приведен пример простейшей нейроной сети, использующейся для классификации изображений из набора данных MNIST.

↳ Скрыто 4 ячейки.

PyTorch